

Resolução Declarativa de Problemas 2

Student 73650

Introdução

Este relatório tem como objetivo **esclarecer** o **desenvolvimento** do trabalho, **facilitar** a **compreensão** do respetivo **output** e **demonstrar** como o **modificar**, de forma a permitir a realização de testes livres. Desta forma, cada capítulo seguinte a este incluirá, pelo menos, estes subcapítulos.

Primeiro Grupo

Desenvolvimento

Definição do problema: O enunciado exige a resolução do mesmo problema proposto no primeiro trabalho, de forma a utilizar a abordagem de *Direct Encoding*. Para atingir esse objetivo, foi necessário definir as três famílias de restrições:

- **At Least 1 (AL1):** Cada peça tem de estar colocada em alguma posição do tabuleiro.

$$\bigvee_{i \in \text{dom}(v)} x_{v,i}$$

- **At Most 1 (AM1):** Uma peça não pode estar em duas ou mais posições do tabuleiro.

$$\neg x_{v,i} \vee \neg x_{v,j}, \quad \forall i, j \in \text{dom}(v), \quad i \neq j$$

- **Conflict Clauses:** Duas peças não podem ocupar posições sobrepostas.

$$\neg x_{v,i} \vee \neg x_{w,j} \quad \text{para cada sobreposição de peça } (v = i, w = j)$$

Definição da variável: Para a abordagem de *Direct Encoding*, e tendo em consideração as restrições definidas anteriormente, foi considerada a seguinte variável:

$$X_{p,x,y} \in \{0, 1\}, \quad p \in [1, n_{\text{Peças}}], \quad x \in [0, \text{maxW} - 1], \quad y \in [0, \text{maxH} - 1]$$

A variável ($X_{p,x,y}$) assume o valor 1 quando a peça (p) tem somente o seu canto superior esquerdo posicionado nas coordenadas $((x,y))$ do tabuleiro, e 0 caso contrário.

Considere-se o seguinte exemplo:

SAT:

```

      1 2 3 4 5
    /-----
1 | 3 3 3 2 2
2 | 3 3 3 2 2
3 | 3 3 3 1 .
  
```

Esta representação foi adotada com o objetivo de reduzir o número de variáveis necessárias no modelo. Em vez de representar explicitamente todas as células ocupadas por uma peça, apenas é armazenada a posição inicial da mesma. Por exemplo, para a peça 2, não é necessário considerar que:

$$X_{2,3,0} = X_{2,4,0} = X_{2,3,1} = X_{2,4,1} = 1$$

Basta representar:

$$X_{2,3,0} = 1$$

A partir desta informação, as restantes posições ocupadas pela peça podem ser inferidas tendo em conta as suas dimensões. Desta forma, evita-se a criação de variáveis redundantes, reduzindo a dimensão do problema e melhorando a eficiência da codificação.

Resultados

Tal como solicitei ao primeiro professor do trabalho, foi mantido o mesmo formato de apresentação dos resultados utilizado anteriormente.

Uma vez que o formato de saída proposto no enunciado do primeiro trabalho corresponde a uma lista de listas, por exemplo:

`[[3, 1, 6, 6, 1], [6, 6, 5, 1, 1]].`

E **optei** por uma **representação** mais **visual** e **intuitiva**, que facilitou a leitura e a análise da solução. Desta forma, em vez de listar apenas os valores, o programa desenha uma grelha correspondente ao problema.

Segue-se um exemplo de saída:

SAT:

```

      1 2 3 4 5 6 7 8 9 10 11 12
    /-----
1 | 5 5 5 5 5 3 3 3 4 4 4 4
2 | 5 5 5 5 5 3 3 3 4 4 4 4
3 | 5 5 5 5 5 3 3 3 4 4 4 4
4 | 5 5 5 5 5 1 2 2 4 4 4 4
5 | 5 5 5 5 5 . 2 2 . . . .
```

Para tal, foi adaptada a função `show_solution_unique` desenvolvida no primeiro trabalho e criada a função `solverPrinter(clauses, x, maxW, maxH)`, integrando os conhecimentos adicionais abordadas na aula laboratorial 6.

Por fim, apresenta-se uma tabela com alguns dos resultados obtidos pelo algoritmo, indicando o valor de N , o tempo de execução e a dimensão da solução ótima encontrada.

N	Tempo de Execução (s)	Dimensão da Solução Ótima
3	0m0,035s	5 × 3
4	0m0,041s	7 × 5
5	0m0,050s	12 × 5
6	0m0,061s	11 × 9
7	0m0,172s	22 × 7
8	0m0,493s	15 × 14
9	0m3,488s	20 × 15
10	0m29,823s	27 × 15
11	0m35,907s	27 × 19
12	8m48,756s	29 × 23

Table 1: Performance obtida para *Direct Encoding*.

O último resultado apresentado na Tabela 1 pode ser consultado no ficheiro (OtherFiles/DirectEncN12.txt), onde se encontra o respetivo *output* completo.

Como Utilizar

Deve-se aceder ao ficheiro Group1/Group1.py.

Para alterar o `minPack(problemN)`, basta ir **final** do **programa** que encontram-se alguns exemplos comentados, e basta retirar o comentário deles ou adicionar um novo caso de teste.

Para determinar quanto tempo demorou, basta executar o programa com 'time' no início da instrução.

Implementações

A função principal responsável pela resolução do problema em função da largura (*width*) e altura (*height*) é a `TilingProblemDirect`. Esta função invoca, separadamente, uma função para a instanciar as variáveis *x*, outra para a criar as cláusulas do tipo AL1, e outra para as cláusulas AM1. As *Conflict Clauses* são criadas diretamente no interior da própria função principal.

Para a instanciação das variáveis, teve-se o cuidado de eliminar previamente casos impossíveis. Por exemplo, num tabuleiro de dimensão 5×5 , ao instanciar variáveis associadas a uma peça de dimensão 3×3 , não faz sentido considerar posições como $(x = 3, y = 2)$, uma vez que a peça ultrapassaria os limites do tabuleiro:

```
def Instantiater(n, maxW, maxH):
    (...)
    for p in range(n): # pieces
        pieceId = p + 1
        for w in range(maxW - pieceId + 1): # Already removing some
                                            # impossible tiling pieces
            for h in range(maxH - pieceId + 1): # Already removing some impossible
                                                # tiling pieces
                x[pieceId, w, h] =
                    pool.id(f"piece {pieceId} has origin at (x:{w}, y:{h})")
    (...)
```

Tanto as cláusulas AL1 como AM1 incluem comentários explicativos ao longo do código, de forma a clarificar o fluxo da criação das cláusulas.

Relativamente às *Conflict Clauses*, foi implementada uma função auxiliar que, com base nas coordenadas possíveis de duas peças, verifica se estas se sobrepõem. Caso seja detetada sobreposição, é adicionada uma cláusula de conflito ao conjunto de cláusulas:

```
(...)
# On each position...
for w1 in range(maxW - id1 + 1):
    for h1 in range(maxH - id1 + 1):
        for w2 in range(maxW - id2 + 1):
            for h2 in range(maxH - id2 + 1):
                # ... check if collide
                if overlap(id1, w1, h1, id2, w2, h2):
                    # Then express Conflict Clauses
                    clauses.append([-x[id1, w1, h1], -x[id2, w2, h2]])
(...)
```

Versão Alternativa do Primeiro Grupo

Desenvolvimento

Apesar de não ser um requisito explícito, considerei este desafio, uma vez que, pela minha intuição, o encoding de *Support Encoding* seria mais eficiente do que o de *Direct Encoding*. Além disso, como apenas a última família difere do encoding Direct, foi possível reutilizar grande parte da implementação anterior.

Definição do problema:

- At Least 1 (AL1): Cada peça tem de estar colocada em alguma posição do tabuleiro.
- At Most 1 (AM1): Uma peça não pode estar em duas ou mais posições do tabuleiro.
- **Support Clauses:** Caso uma peça ocupe determinada posição as outras peças não podem ficar nessa mesma posição:

$$\neg x_{w,j} \vee x_{v,i}, \quad i \in S_{v,j,w}$$

Resultados

Apresenta-se uma tabela com alguns dos resultados obtidos pelo algoritmo, indicando o valor de N , o tempo de execução e a dimensão da solução ótima encontrada.

N	Tempo de Execução (s)	Dimensão da Solução Ótima
3	0m0,036s	5 x 3
4	0m0,048s	7 x 5
5	0m0,045s	12 x 5
6	0m0,068s	11 x 9
7	0m0,237s	22 x 7
8	0m0,573s	15 x 14
9	0m5,583s	20 x 15
10	1m49,520s	27 x 15
11	4m37,594s	27 x 19
12	> 51m38,229s Aborted	NaN

Table 2: Performance obtida para *Support Encoding*.

Como utilizar

Deve-se aceder ao ficheiro `otherFiles/SupEncond.py`.

Para alterar o `minPack(problemN)`, basta ir **final** do **programa** que encontram-se alguns exemplos comentados, e basta retirar o comentário deles ou adicionar um novo caso de teste.

Para determinar quanto tempo demorou, basta executar o programa com 'time' no início da instrução.

Implementações

A principal diferença em relação ao *Direct Encoding* consiste na forma como as restrições são criadas. Aqui as cláusulas de conflito são adicionadas caso não seja detetada uma sobreposição entre as peças.

```
for x1 in range(maxW - id1 + 1): # width
    for y1 in range(maxH - id1 + 1): # height
        clause = [-x[id1, x1, y1]] # (¬x[piece1, (Pos1)] V (...))

        for x2 in range(maxW - id2 + 1): # width
            for y2 in range(maxH - id2 + 1): # height
                # If not overlapping then can be support var
                if not overlap(id1, x1, y1, id2, x2, y2):
                    clause.append(x[id2, x2, y2])
```

Segundo Grupo

Desenvolvimento

Definição do problema: Neste grupo, o enunciado exige a resolução do problema recorrendo a uma abordagem em linguagem imperativa. Para tal, foi necessário identificar as restrições fundamentais do problema:

1. Cada peça tem no **minimo uma** posição do tabuleiro.
2. Cada peça tem no **máximo uma** posição do tabuleiro.

3. Uma peça **não pode** ser colocada fora dos limites do tabuleiro.
4. Duas peças **não podem** ocupar posições sobrepostas.
5. (Opcional) Utilizar *Multi-shot solving*: recorrendo a átomos *externals* para controlar as variáveis que alteram o estado entre execuções.

Definição da variável: Para a modelação do problema com a API de controlo, a abordagem inicial seria utilizar a mesma dos predicados do *Direct Encoding*:

$$X_{p,x,y} \equiv \text{place}(P, X, Y)$$

No entanto, após alguns testes, verificou-se por causa do uso de *Multi-shot solving* fazia com que o programa lançasse um erro do tipo "*redefinition of atom*" em vez de reutilizar a variável de escolha. Desta forma, para contornar esta limitação, desenhou-se uma variável com o contexto do tabuleiro:

$$X_{w,h,p,x,y} \in [0, 1], \quad w \in [0, \text{maxW} - 1], \quad h \in [0, \text{maxH} - 1], \quad p \in [0, n_{\text{NPeças}} - 1],$$

$$x \in [0, \text{maxW} - 1], \quad y \in [0, \text{maxH} - 1]$$

Desta forma, a cada configuração de tabuleiro são criados átomos **distintos**, o que **evita colisões** entre iterações e permitem realizar a pesquisa incremental de forma segura e eficiente.

Resultados

Apresenta-se uma tabela com alguns dos resultados obtidos pelo algoritmo, indicando o valor de N , o tempo de execução e a dimensão da solução ótima encontrada.

N	Tempo de Execução (s)	Dimensão da Solução Ótima
3	0m0,036s	5 x 3
4	0m0,044s	7 x 5
5	0m0,052s	12 x 5
6	0m0,107s	11 x 9
7	0m0,475s	22 x 7
8	0m1,213s	15 x 14
9	0m8,574s	20 x 15
10	34m44,657s	27 x 15

Table 3: Performance obtida para *Imperative*.

Como utilizar

Deve-se aceder ao ficheiro `Group2/script.py`.

Para alterar o `minPack(problemN)`, basta ir **final** do **programa** que encontram-se alguns exemplos comentados, e basta retirar o comentário deles ou adicionar um novo caso de teste.

Para determinar quanto tempo demorou, basta executar o programa com `'time'` no início da instrução.

Implementações

O número de peças é parametrizado através do seguinte facto:

```
#program base.
(...)
piece(1..n).
```

As restrições correspondentes às condições 1 e 2, são implementadas através da seguinte regra:

```
1 { place(w, h, P, X, Y) : position(w, h, X, Y) } 1 :- piece(P), boardSize(w, h).
```

A condição 3 é garantida pelas seguintes restrições:

```
:- place(w, h, P, X, _Y), boardSize(w, h), w < P + X.
:- place(w, h, P, _X, Y), boardSize(w, h), h < P + Y.
```

A condição 4 é assegurada através da restrição seguinte

```
:- place(w, h, P1, X1, Y1), place(w, h, P2, X2, Y2), P1 < P2,
   X1 < X2 + P2, X2 < X1 + P1,
   Y1 < Y2 + P2, Y2 < Y1 + P1.
```

Por fim, para suportar *Multi-shot Solving* (Condição 5), foi utilizado o átomo externo `boardSize(w,h)`:

```
#external boardSize(w, h).
```

Terceiro Grupo

Comparação de Resultados

A seguir pode ser vista uma tabela que agrega todos os valores obtidos ao longo dos vários grupos:

N	Tempo Direct Encoding	Tempo Support Encoding	Tempo Imperative Control
3	0m0,035s	0m0,036s	0m0,036s
4	0m0,041s	0m0,048s	0m0,044s
5	0m0,050s	0m0,045s	0m0,052s
6	0m0,061s	0m0,068s	0m0,107s
7	0m0,172s	0m0,237s	0m0,475s
8	0m0,493s	0m0,573s	0m1,213s
9	0m3,488s	0m5,583s	0m8,574s
10	0m29,823s	1m49,520s	34m44,657s
11	0m35,907s	4m37,594s	- - -
12	8m48,756s	> 51m38,229s <u>Aborted</u>	- - -

Table 4: Comparação de valores obtidos neste segundo trabalho

Tal como se pode observar na Tabela 4, o *Direct Encoding* apresenta o melhor desempenho global entre as três abordagens analisadas. Em segundo lugar temos o *Support Encoding*, enquanto a abordagem de *Imperative Control* apresenta os tempos de execução mais elevados.

O *Direct Encoding* foi capaz de obter resultados até $N = 12$. O *Support Encoding* conseguiu produzir resultados até $N = 11$. Por sua vez, a abordagem de *Imperative Control* apenas permitiu obter resultados até $N = 10$.

Estes resultados sugerem que o *Direct Encoding* constitui a abordagem mais eficiente para o **problema em estudo**.

Já a seguir, se encontra uma tabela que compara os melhores valores entre os dois trabalhos:

N	Tempo Direct Encoding	Terceira Parte do Primeiro Trabalho
11	0m35,907s	0m0,047s
12	8m48,756s	0m0,106s
13	- - -	0m0,543s
17	- - -	0m59,716s

Table 5: Comparação dos melhores valores dos dois trabalhos

Tal como se pode observar na Tabela 5, a performance obtida na terceira parte do primeiro trabalho são significativamente **superiores** aos alcançados pelo *Direct Encoding*. Enquanto esta última abordagem apenas conseguiu resolver instâncias até $N = 12$, a solução desenvolvida no primeiro trabalho (*Constraint Programming*) foi capaz de obter resultados até $N = 17$.

Reflexões

Relativamente aos resultados apresentados na Tabela 4, considerei inicialmente, que o *Support Encoding* apresentasse melhor desempenho do que o *Direct Encoding*, uma vez que introduz cláusulas adicionais que podiam facilitar o processo de procura. Da mesma forma, também se esperava que a abordagem de *Imperative Control*, devido à utilização de *Multi-shot Solving*. No entanto, os resultados obtidos não confirmaram estas expectativas, tendo o *Direct Encoding* demonstrado melhor desempenho dos três.

No que diz respeito aos resultados da Tabela 5, já considerava que o primeiro projeto, pudesse apresentar resultados superiores. O trabalho anterior teve o apoio de uma documentação que apresentava implementações bastante sofisticadas, cuja aplicação se refletiu com clareza em melhores resultados.

Uso de LLMs

O único uso de LLMs neste trabalho foi exclusivamente para a **melhoria e revisão do texto**, uma vez que tenho algumas dificuldades associadas à dislexia. Não foi utilizada qualquer LLM para a criação de código.